

Multiple Linear Regression: The Normal Equation

Contents

1. Overview	2
2. The model in matrix form	2
3. The error in matrix form	3
4. Expanding the error	4
5. Setting the derivative to zero	4
6. The normal equation	5
6.1 A worked example	5
7. The cost of the inverse	5
8. Summary	7
9. Key terms	7

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Regression problems ([Note 3](#)); Ordinary least squares (closed form) ([Note 51](#)); Simple linear regression ([Note 51](#)).
- **Leads to:** Polynomial regression ([Note 61](#)); Ridge regression ([Note 63](#)).
- **Compare with:** Gradient descent ([Note 57](#)); Moore-Penrose pseudo-inverse ([Note 613](#)); ANN for regression ([Note 1013](#)).

1. Overview

Key point: Written with matrices, the error of multiple linear regression has one formula for all the coefficients at once: $\beta = (X^T X)^{-1} X^T y$, the normal equation.

For simple linear regression we found two formulas, one for m and one for b . With m input columns there are $m + 1$ coefficients, and writing a separate formula for each is hopeless.

Matrices solve this. We write all the data, all the predictions and all the coefficients as matrices, and the same steps as before (write the error, differentiate, set to zero) give one formula for every coefficient at once:

$$\beta = (X^T X)^{-1} X^T y$$

This Note builds it step by step. The next Note codes it from scratch. The derivation uses a few rules about matrices; each is stated where it is used.

2. The model in matrix form

Key point: Put the data in a matrix X with an extra first column of 1s, and the coefficients in a vector β . Then all predictions at once are $\hat{y} = X\beta$.

For one data point i with inputs $x_{i1}, \dots, x_{im}$, the model predicts

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_m x_{im}$$

Writing this for every row gives n equations. Matrices write them all at once (Figure 1).

$$\underbrace{\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix}}_{\hat{y}: n \times 1} = \underbrace{\begin{bmatrix} \textcolor{brown}{1} & x_{11} & x_{12} & \cdots & x_{1m} \\ \textcolor{brown}{1} & x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ \textcolor{brown}{1} & x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix}}_{X: n \times (m+1)} \underbrace{\begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_m \end{bmatrix}}_{\beta: (m+1) \times 1}$$

an extra column of 1s carries the intercept β_0

rows: data points (n) columns: a 1, then the m inputs β : the $m + 1$ numbers to find

Figure 1: All predictions as one matrix product

- X holds the data: one row per data point, one column per input, plus a first column of 1s. The 1s multiply β_0 , so the intercept is treated like any other coefficient.
- β holds the $m + 1$ coefficients, β_0 to β_m .
- $\hat{y}$ holds the n predictions.

Multiplying a row of X by β gives exactly $\beta_0 \cdot 1 + \beta_1 x_{i1} + \cdots + \beta_m x_{im}$, one prediction. So all predictions together are

$$\hat{y} = X\beta$$

Extra: The shapes must fit: X is $n \times (m + 1)$ and β is $(m + 1) \times 1$. The inner sizes match, and the product is $n \times 1$, one prediction per row.

3. The error in matrix form

Key point: The vector of errors is $e = y - X\beta$. The sum of squared errors is $e^T e$.

The errors of all points form a vector:

$$e = y - \hat{y} = y - X\beta$$

The sum of squared errors, $E = e_1^2 + e_2^2 + \cdots + e_n^2$, is a row times a column: the transpose e^T (the same numbers written as a row) multiplied by e .

$$E = e^T e = (y - X\beta)^T (y - X\beta)$$

With numbers: if $e = (0.2, -0.1, 0.3)$, then $e^T e = 0.04 + 0.01 + 0.09 = 0.14$.

4. Expanding the error

Key point: Multiplying out the brackets gives $E = y^T y - 2y^T X\beta + \beta^T X^T X\beta$.

Two rules about transposes are needed:

- **The transpose of a product reverses the order:** $(AB)^T = B^T A^T$. So $(X\beta)^T = \beta^T X^T$.
- **A single number is its own transpose.** The terms $y^T X\beta$ and $\beta^T X^T y$ are each one number (1×1), and each is the transpose of the other, so they are equal.

Multiplying out:

$$E = y^T y - y^T X\beta - \beta^T X^T y + \beta^T X^T X\beta$$

The two middle terms are equal, so they combine:

$$E = y^T y - 2y^T X\beta + \beta^T X^T X\beta$$

5. Setting the derivative to zero

Key point: Differentiating E with respect to β and setting it to zero gives $X^T X\beta = X^T y$.

As in simple linear regression, the best coefficients sit at the bottom of the error bowl, where the derivative is zero. Now the derivative is taken with respect to the whole vector β : it is a vector of $m + 1$ partial derivatives, one per coefficient, and all must be zero.

Two rules of **matrix calculus** do the work, the matrix versions of “the derivative of ax is a ” and “the derivative of ax^2 is $2ax$ ”:

Term	Derivative with respect to β
$y^T y$ (no β in it)	0
$2y^T X\beta$ (like $a\beta$)	$2X^T y$
$\beta^T X^T X\beta$ (like $a\beta^2$)	$2X^T X\beta$

The last rule holds because $X^T X$ is symmetric. So

$$\frac{\partial E}{\partial \beta} = -2X^T y + 2X^T X\beta = 0$$

Dividing by 2 and moving one term across:

$$X^T X\beta = X^T y$$

These are the **normal equations**: $m + 1$ equations, one for each coefficient.

6. The normal equation

Key point: Multiplying both sides by the inverse of $X^T X$ isolates β .

To get β alone we need to “divide” by $X^T X$. Matrices have no division; instead we multiply by the **inverse** $(X^T X)^{-1}$, the matrix that undoes $X^T X$ (their product is the identity matrix, which changes nothing).

$$\beta = (X^T X)^{-1} X^T y$$

This is the **normal equation**: a closed-form solution for all coefficients of multiple linear regression at once. scikit-learn’s `LinearRegression` computes this solution (with a numerically safer method than a literal inverse).

6.1 A worked example

Key point: On four students with one input, the normal equation gives the same slope and intercept as the simple formulas.

Take the first four students of the placement data, with CGPA 6.89, 5.12, 7.82, 7.42 and packages 3.26, 1.98, 3.25, 3.67.

1. **Build X :** a column of four 1s next to the four CGPAs.
2. **Compute the pieces:**

$$X^T X = \begin{bmatrix} 4 & 27.25 \\ 27.25 & 189.90 \end{bmatrix} \quad X^T y = \begin{bmatrix} 12.16 \\ 85.25 \end{bmatrix}$$

The top-left 4 counts the rows; 27.25 is the sum of the CGPAs; 189.90 is the sum of their squares; 12.16 is the sum of the packages.

3. **Invert and multiply:**

$$\beta = \begin{bmatrix} 11.158 & -1.601 \\ -1.601 & 0.235 \end{bmatrix} \begin{bmatrix} 12.16 \\ 85.25 \end{bmatrix} = \begin{bmatrix} -0.81 \\ 0.57 \end{bmatrix}$$

So $\beta_0 = -0.81$ (intercept) and $\beta_1 = 0.57$ (slope): exactly what the simple linear regression formulas give for these four points. The simple formulas are the normal equation with a single input.

7. The cost of the inverse

Key point: Inverting $X^T X$ takes time that grows roughly with the cube of the number of columns. With very many columns, gradient descent is used instead.

$X^T X$ is a square matrix with one row and one column per coefficient, $(m + 1) \times (m + 1)$. Inverting an $m \times m$ matrix takes on the order of m^3 operations: doubling the columns makes the work about 8 times larger.

Figure 2 measures it on this computer.

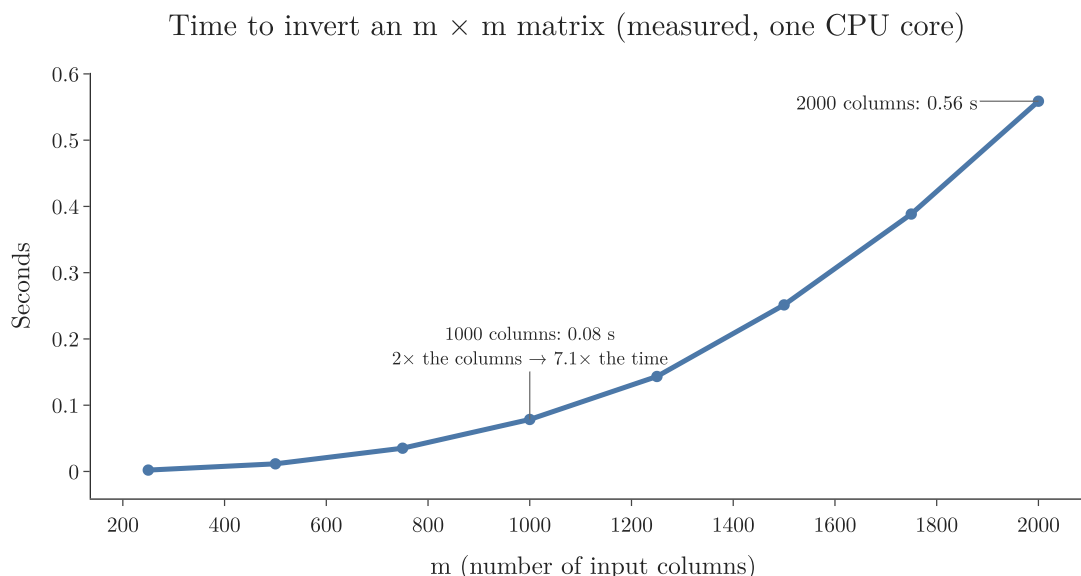


Figure 2: Time to invert a matrix as its size grows

From 1,000 to 2,000 columns, the time grows about 7 times, from 0.08 to 0.59 seconds. With tens of thousands of columns, as with text or image data, the inverse becomes very slow and memory-hungry.

That is why there is a second method, **gradient descent**: it does not compute any inverse, but approaches the best coefficients step by step. Its answer is very close to the normal equation's. In scikit-learn:

- `LinearRegression` uses the closed-form (OLS) solution;
- `SGDRegressor` uses gradient descent.

For most tabular data the number of columns is small, and `LinearRegression` is the usual choice. Gradient descent gets its own Notes next.

Extra: $X^T X$ has no inverse when one input column can be built exactly from others (multicollinearity, as in the dummy variable trap of the one-hot encoding Note). Then the normal equation has no unique answer. Libraries handle this with a “pseudo-inverse” or with regularisation, both covered later.

8. Summary

Step	Formula
Predictions	$\hat{y} = X\beta$ (X has a first column of 1s)
Errors	$e = y - X\beta$
Sum of squared errors	$E = e^T e$
Expanded	$E = y^T y - 2y^T X\beta + \beta^T X^T X\beta$
Derivative set to zero	$X^T X\beta = X^T y$
Normal equation	$\beta = (X^T X)^{-1} X^T y$

- A column of 1s in X lets the intercept be treated like any other coefficient.
- One formula gives all $m + 1$ coefficients at once; with one input it reduces to the simple formulas.
- The inverse costs about m^3 operations, so very wide data uses gradient descent instead.

9. Key terms

Term	Meaning
Design matrix (X)	The data as a matrix, one row per data point, with a first column of 1s for the intercept
Coefficient vector (β)	All the coefficients of the model, β_0 to β_m , as one column
Matrix calculus	Rules for differentiating expressions with vectors and matrices
Normal equations	$X^T X\beta = X^T y$: the conditions that the best coefficients satisfy
Normal equation	$\beta = (X^T X)^{-1} X^T y$: the closed-form solution of linear regression
Inverse matrix	The matrix that undoes another: their product is the identity matrix
Identity matrix	The square matrix with 1s on the diagonal and 0s elsewhere; multiplying by it changes nothing